The University of Texas at Austin

McCombs School of Business

RM 294: Optimization II

Project 3 Report - Reinforcement Learning for Connect Four Strategy Development

Team Members:

Jacob Embleton, Abhiroop Kumar, Aileen Li, Riju Hariharan

April 27th, 2026

Executive Summary

This project studies whether reinforcement learning can improve Connect-4 strategies beyond pretrained neural network policies. We evaluate two reinforcement learning approaches, Policy Gradient (PG) and Deep Q-Networks (DQN), to assess whether these methods can produce stronger computer-controlled opponents and whether reinforcement learning expertise would be valuable in a gaming setting.

Before training reinforcement learning agents, we first evaluate the starting policies to establish how strong they already are and to create a benchmark for later comparison. The baseline results show that the starting policies already perform well against benchmark opponents and also exhibit meaningful differences when playing against one another. This makes reinforcement learning improvement more challenging, since the objective is not to learn basic gameplay from scratch, but to improve an already capable policy.

The reinforcement learning experiments show differing outcomes across methods. Policy Gradient delivered measurable improvement over the baseline model within the project's training budget, while Deep Q-Networks showed partial competence but required substantially more training to become competitive against strong opponents. These results suggest reinforcement learning can add value, but that implementation quality, training design, and computational budget matter critically.

Overall, the findings support an incremental strategy in which strong supervised policies are refined through reinforcement learning rather than replaced by RL from scratch. The results also suggest that reinforcement learning expertise could be beneficial, particularly for designing stable self-play training pipelines, improving convergence, and scaling these methods toward production-quality game agents.

The sections below describe the problem setup, baseline evaluation, reinforcement learning experiments, comparative results, and the practical implications of these findings for deploying RL-based game agents.

Problem Formulation

We framed Connect-4 as a sequential decision-making problem in which an agent repeatedly observes the game board, chooses an action, and receives rewards based on eventual game outcomes. This structure makes the problem naturally suited to reinforcement learning.

The problem was modeled through four key components.

1. **State Representation**
   The state is the current 6 × 7 board configuration. For model inputs, the board is represented from the active player's perspective using a two-channel encoding, one channel representing the player's pieces and the other representing the opponent's pieces. This representation allows the same policy framework to be applied regardless of which side is moving.

2. **Action Space**
   At each turn, the agent chooses one of seven legal columns into which a piece may be dropped. Illegal actions, such as selecting a full column, are excluded from the feasible action set.

3. **State Transition**
   After an action is selected, the board updates according to the game rules and play passes to the opponent. Because each decision affects future board states and future opportunities, the problem has the sequential dependence central to reinforcement learning.

4. **Reward Structure**
   Rewards are assigned only when a game ends:
   - Win = +1
   - Loss = –1
   - Draw = 0

   This reward structure focuses learning on long-term success rather than short-term moves.

Computational Considerations

Even though there are only seven actions at each turn, Connect-4 has a large state space and long planning horizons. Because the game tree grows exponentially with search depth, exact optimization becomes computationally prohibitive, motivating approximate methods such as neural policies and reinforcement learning.

Experimental Setup and Reproducibility

To compare policies fairly, all agents were evaluated under the same match framework.

For each matchup:

- who moves first alternates across games
- win rates and draws are recorded
- average game length is tracked
- performance as first and second player is considered

Alternating move order is important because moving first can be an advantage in Connect-4, so controlling for that helps make comparisons more meaningful.

One teammate model was excluded from evaluation because its saved model caused repeated deserialization issues in the TensorFlow environment. To keep the experiments reproducible, the analysis was carried out using the three models that loaded reliably.

Before reinforcement learning methods were trained, this evaluation framework was used to establish the strength of the starting policies.

Base Model Selection

One policy was selected as the starting point for reinforcement learning improvement. The selection was based on baseline head-to-head performance against peer policies and benchmark opponents, with emphasis on consistency and overall strength across matchups.

The results of this baseline evaluation are presented in the next section, where the selected base policy is justified.

Choosing a strong initial policy makes the reinforcement learning task more demanding, but also more meaningful. This is because improvements must exceed an already capable baseline rather than simply learn basic tactics.

Baseline Evaluation

Purpose of Baseline Analysis

Baseline testing was conducted to answer three questions:

1. Do the starting policies exhibit competent gameplay behavior?
2. How do the policies compare against one another?
3. Is there evidence that reinforcement learning may add value beyond the starting policies?

These results establish the reference point against which later reinforcement learning performance is evaluated.

Baseline Head-to-Head Performance

A round-robin tournament was conducted among the learned policies, random play, and a tactical-random benchmark opponent.

Table 1. Pairwise Win Rates Among Evaluated Agents

	Jake_M1_original	Abhiroop_CNN	Riju_CNN	Random	Tactical_Random
Jake_M1_original	NaN	0.5	0.0	1.0	1.00
Abhiroop_CNN	0.0	NaN	0.0	1.0	0.98
Riju_CNN	1.0	1.0	NaN	1.0	0.98
Random	0.0	0.0	0.0	NaN	0.08
Tactical_Random	0.0	0.0	0.0	1.0	NaN

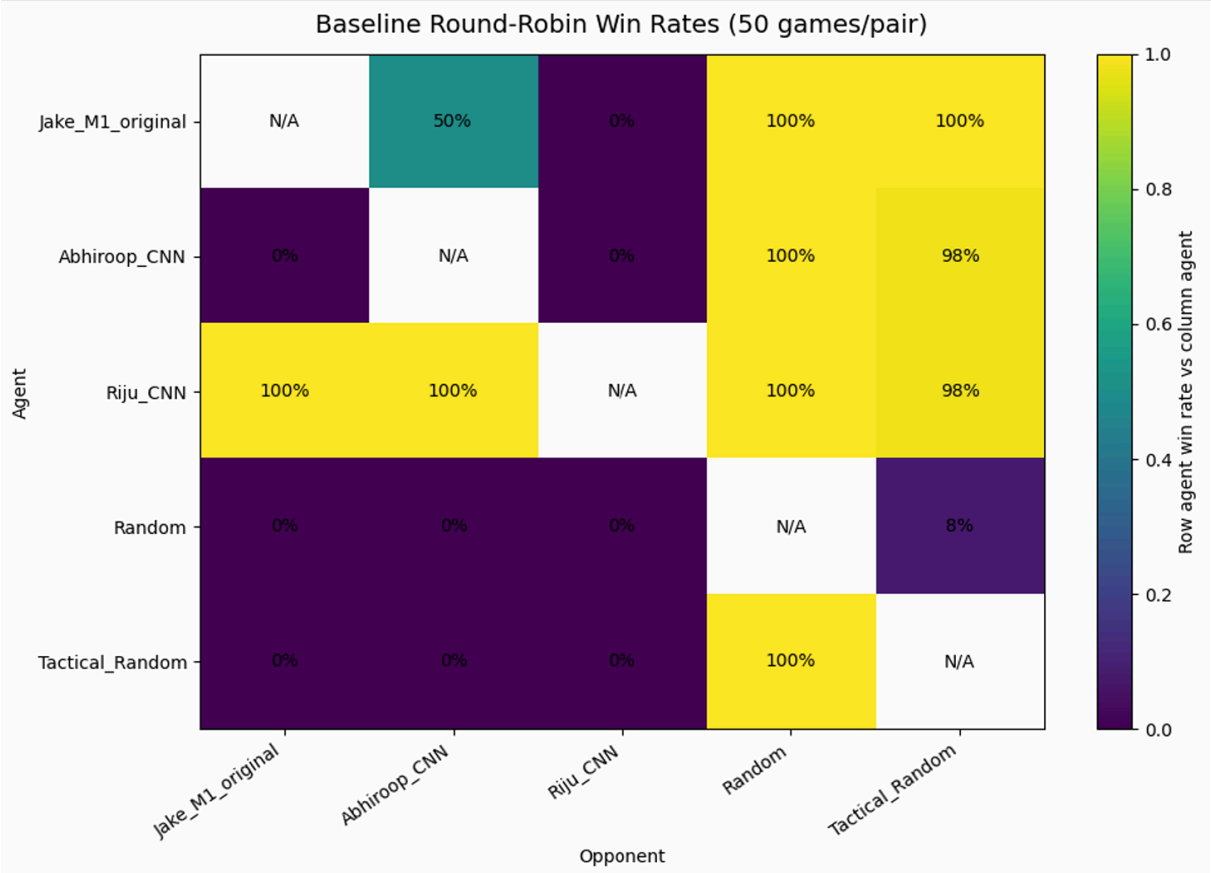

Figure 1. Round-Robin Win Rate Heatmap Across Baseline Agents

Table 1 and **Figure 1** summarize results from the same round-robin evaluation, with **Table 1** providing the numerical win rates and **Figure 1** giving a visual summary of those pairwise relationships.

Several patterns emerge from **Table 1** and **Figure 1**. First, the learned policies consistently outperform the random and tactical-random benchmark agents. This suggests they have learned structured gameplay behavior beyond random or purely heuristic play.

Second, the results show meaningful differences even among the learned policies themselves. Although all three models are competitive, they do not perform identically. The results suggest meaningful differences in relative policy strength, with Riju_CNN emerging as the strongest performer.

Figure 1 also shows the tactical-random benchmark is particularly informative. As shown in **Table 1**, Riju_CNN achieved 100% win rates against both teammate models, 100% against the random benchmark, and 98% against Tactical_Random, supporting its selection as the base policy. Since this opponent incorporates immediate win and blocking logic, strong performance against it suggests the learned policies have captured more than basic tactical reactions.

These results also help justify the choice of base policy for reinforcement learning. Based on the head-to-head results shown in **Table 1** and **Figure 1**, Riju_CNN was selected as the model to improve because it showed the strongest and most consistent performance across matchups.

Benchmark Validation Against Random Play

Table 2. Performance Against Random Opponent

	agent_a	a_win_rate	draw_rate	avg_moves	a_wins_as_p1	a_wins_as_p2
0	Jake_M1_original	1.0	0.0	9.40	50	50
1	Abhiroop_CNN	1.0	0.0	9.96	50	50
2	Riju_CNN	1.0	0.0	9.96	50	50

Table 2 provides an additional validation of policy strength against unstructured play. All three policies achieve perfect win rates against the random opponent, confirming that each has learned meaningful gameplay behavior. The relatively short game lengths are also notable. As shown in **Table 2**, wins were achieved in roughly 9-10 moves on average, suggesting these models exploit favorable positions efficiently rather than relying on long games to eventually win. This means that models are not simply winning eventually, but often exploiting favorable positions efficiently. Another useful takeaway is the consistency in performance regardless of move order. Strong results both moving first and second suggest these policies are not relying solely on first-player advantage.

While **Table 1** and **Figure 1** highlight differences among strong policies, **Table 2** shows all of them exceed a clear minimum level of competence. Thus, these results provide confidence that the baseline policies offer a credible starting point for reinforcement learning.

Baseline Results and Discussion

We can conclude the following from the baseline analysis.

First, the starting policies already exhibit strong tactical competence against both benchmark and learned opponents. This means reinforcement learning is not beginning from weak play, but attempting to improve upon an already capable strategy.

Second, because the selected base policy already performs strongly against peer policies and benchmark opponents, improvements from later reinforcement learning should be interpreted as incremental gains over a strong benchmark.

Third, the results suggest imitation-based policies provide a strong benchmark for assessing whether reinforcement learning produces enough incremental improvement to justify its added complexity.

This makes the reinforcement learning phase more meaningful. Improvement is not assumed, it has to be demonstrated. This also raises a practical business question explored in later sections, whether improvements from reinforcement learning are large enough to justify specialized RL expertise. These findings motivate the reinforcement learning experiments that follow.

Policy Gradient Methodology

Policy Gradient (PG) training improves the base model M1 (Riju_CNN) by sampling moves from its own probability distribution during self-play, then nudging that distribution toward actions that led to wins and away from actions that led to losses. Unlike supervised learning, which imitates expert moves, PG learns from outcomes alone - making it well-suited for environments where the "correct" move is not always known but the result of a full game is.

Self-play setup. Each training game begins with 0–6 fully random opening moves to diversify starting positions. After the random opening, M1 is randomly assigned to play either first or second, and a randomly selected opponent from the opponent pool plays the other side. M1 selects moves stochastically (sampling from its softmax output rather than taking the argmax) with a 10% uniform mix added to ensure additional exploration. Only the moves M1 itself chose are recorded as training data; the random opening moves are excluded.

Opponent Pool

The pool starts with three Project 1 models (Riju_CNN, Jake_M1_original, Abhiroop_CNN), each wrapped as a "strong" opponent that consults force_win and block_loss tactical helpers. Every 15 iterations, a snapshot of the current M1 is added to the pool, with the pool capped at four snapshots to keep evaluation tractable while still exposing M1 to past versions of itself.

Adversarial Training Logic

A deliberate asymmetry is used: opponents in the pool are allowed to use force_win and block_loss to play tactically optimal moves, while M1 itself is restricted to legal-move masking only. This mirrors the project specification: by forcing M1 to learn tactics through gradient descent rather than handing them rules, the resulting policy generalizes better, the same principle behind adversarial training and dropout.

Discounted Reward and Update Rule

When a game ends, the terminal outcome (+1 win, −1 loss, 0 draw) is discounted backward through M1's move sequence using γ = 0.99. Each move is associated with a (board, action, discounted reward) triplet and added to a 5,000-entry replay buffer. At each training iteration, a fixed batch of 128 triplets is sampled and used for a single REINFORCE-style update:

$$L = -E[A_t \cdot log\, \pi(a_t \mid s_t)],$$

Where A_t is discounted advantage for action step at step t (normalizes within in batch), $\pi(a_t \mid s_t)$ is the model's probability of the chosen action given the board state, and $E[]$ is the empirical mean over a fixed batch of 128 (state, action, advantage) triplets.

Advantages are normalized across the batch when variance is non-trivial. The fixed batch size matters in practice. The project specification flags that variable batch sizes cause TensorFlow to retrace the graph and slow training drastically.

Policy Gradient Training Experiments

The PG agent was trained with the following configuration:

Table 3. Policy Gradient Training Configuration

Hyperparameter	Value
Iterations	60
Games per iteration	8
Batch size	128

Optimizer	Adam
Learning rate	1e-4
Discount factor γ	0.99
Replay buffer cap	5,000 triplets
M1 action selection	Stochastic + 10% uniform mix
Opponent strength	Force_win + block_loss enabled
Snapshot cadence	Every 15 iterations
Evaluation cadence	Every 5 iterations
Eval games per checkpoint	40 (vs M1 and Random)

The evaluation protocol uses argmax action selection (not stochastic) when measuring performance, since deterministic play is what would be deployed at inference time. Win rates are computed over alternating-first-player matches against two reference points: the original (untrained) M1 to detect improvement, and a Random agent to confirm baseline competence is maintained.

Policy Gradient Results and Diagnostics

Training Trajectory

The training-outcome curve (**Figure 2**) shows the rolling mean win/loss outcome over the 60 iterations. The curve fluctuates between approximately −0.5 and +0.2, reflecting the inherently noisy nature of PG self-play - particularly when opponents in the pool use tactical overrides that punish exploratory moves. Outcomes are not expected to climb monotonically because the opponent distribution itself shifts as snapshots are added.

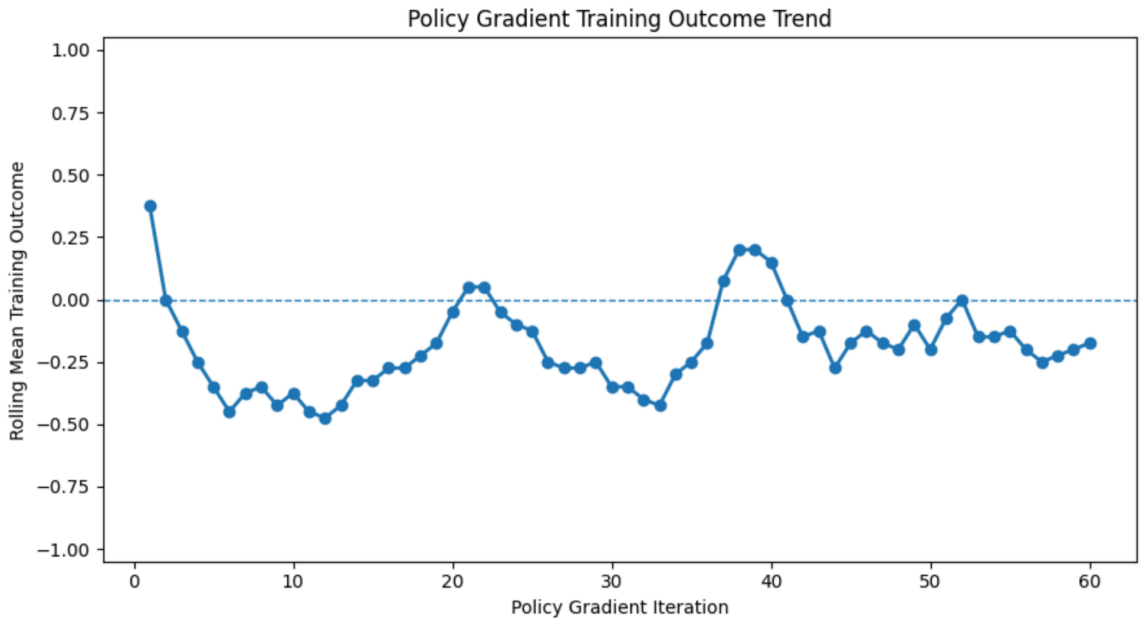

Figure 2.Policy Gradient Training Outcome Trend

Win Rate against the original M1

The headline result (**Figure 3**) shows that the PG-improved model reached a 100% win rate at several evaluation checkpoints between 35 and 55, before regressing to 50% at iteration 60. Throughout training, the PG model maintains a 100% win rate against a Random opponent, confirming that improvements against M1 do not come at the cost of basic tactical competence.

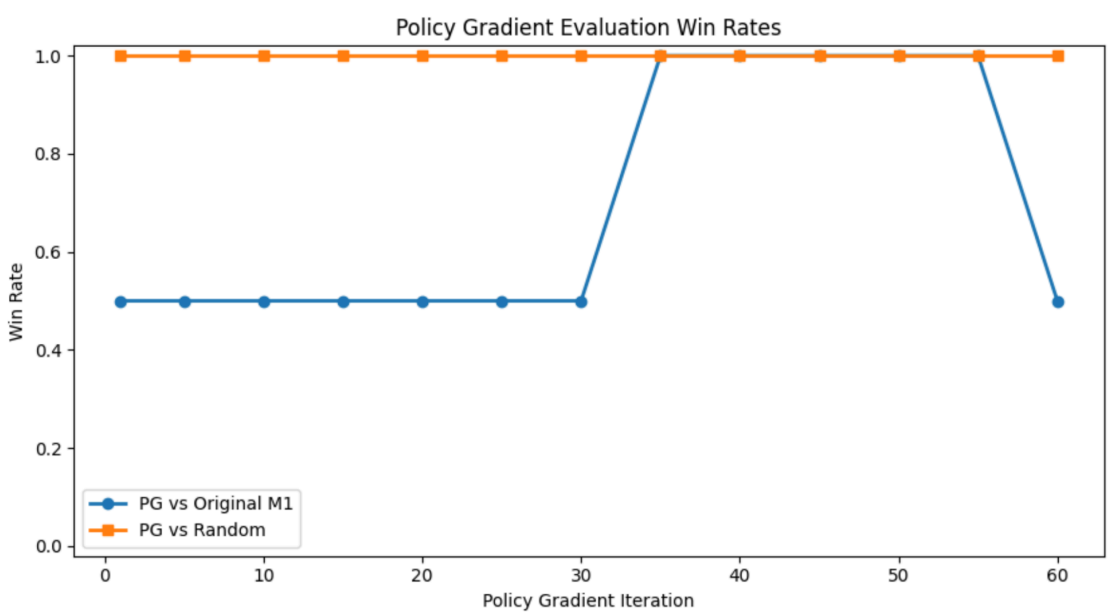

Figure 3. Policy Gradient Evaluation Win Rates

Loss Curve

The training loss (**Figure 4**) is consistently negative - expected for a REINFORCE objective where the loss is −advantage · log π - and varies between roughly −0.1 and −1.2. Spikes in magnitude correspond to iterations where the advantage signal is large (decisive wins or losses), which is normal behavior. There is no evidence of divergence or collapse.

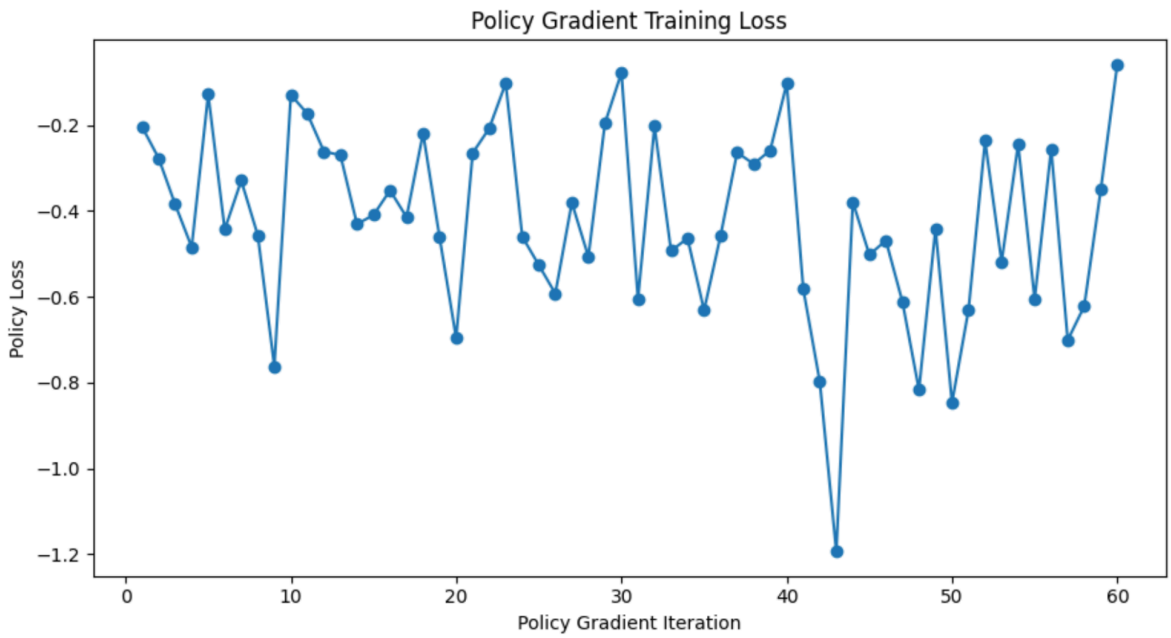

Figure 4.Policy Gradient Training Loss

Stability

Variance across iterations is the dominant feature of PG training. A single losing game can attribute negative advantage to a long sequence of M1 moves, some of which may have been individually correct. This is a known characteristic of REINFORCE-style methods and is partially mitigated here by advantage normalization within each batch and by the replay buffer smoothing the data distribution. Even with these mitigations, results between adjacent evaluation points can swing by 50 percentage points.

In the final round-robin (100 games per pair), the PG-improved model achieved a 50% win rate against the original M1 and 100% against Random, Abhiroop_CNN, and the DQN agent. Against Jake_M1_original it tied at 50%.

Policy Gradient Strengths and Limitations

Strengths

PG integrates cleanly with the existing supervised policy network. Because the model architecture and output format are unchanged, the PG-trained model can be dropped directly

into any pipeline that expected the original M1 - no wrapper changes, no new inference logic. This is a meaningful operational advantage if the company is already using policy networks in production.

PG also produces visible learning signal within a relatively small training budget. By iteration 35, the agent had matched or exceeded the original M1; by comparison, the DQN agent in the same project failed to beat M1 a single time across 300 episodes. For a company evaluating RL feasibility, PG offered the clearest evidence of improvement within the project's training budget.

Limitations

The principal weakness is variance. The win-rate curve against M1 swings between 50% and 100% across evaluation points, and the final iteration regresses to 50%. In a production setting, this means the choice of which iteration to ship matters - the agent at iteration 50 is meaningfully different from the agent at iteration 60, and the difference is not predictable from training-side metrics alone. Best-checkpoint selection based on held-out evaluation games is essential.

Sample efficiency is also a concern. The 60-iteration run used roughly 480 self-play games, and the spec explicitly suggests scaling to 200+ iterations if time allows. Training to a stable, dominant policy likely requires an order of magnitude more games.

Finally, the 50% win rate against Jake_M1_original in the final round-robin suggests the agent has not generalized beyond the opponents it trained against most often. A larger and more diverse opponent pool would likely close this gap.

Deep Q-Network Methodology

The DQN approach learns to evaluate Connect-4 positions by estimating the expected future reward of each possible column selection. Unlike Policy Gradient, which directly adjusts move probabilities, DQN learns a value function that answers: given this board position, what is the long-term payoff of each action? The agent then selects the highest-valued legal move at inference time.

Network Architecture

The DQN shares the same convolutional backbone as the Policy Gradient network, using a 6x7x2 board encoding as input (two channels: one for the DQN player's pieces, one for the opponent's pieces). The critical architectural difference is the output layer: the PG network terminates with a softmax over seven columns to produce a probability distribution, while the DQN terminates with a tanh activation over seven columns to produce bounded Q-values in the range [-1, +1]. This range was chosen to match the reward structure of the game, where +1 represents a win, -1 a loss, and 0 a draw.

The full layer stack is as follows:

Table 4: DQN Network Architecture

Layer	Configuration	Activation	Output Shape
Input	6 × 7 × 2 board encoding	N/A	6,7,2
Conv2D	64 filters, 4 × 4 kernel, same padding	ReLU	(6,7,64)
Conv2D	64 filters, 4 × 4 kernel, same padding	ReLU	(6,7,64)
Flatten	N/A	N/A	2688
Dense	128 units	ReLU	128
Dense	64 units	ReLU	64
Output (Dense)	7 units, one per column	tanh	7

State and Action Representation

The state is the board position from the DQN player's perspective, encoded as a two-channel (6x7x2) array. An important design choice concerns how the next state is defined: the next state is the board position on the DQN's next turn, not the board immediately after the DQN's move. This means the opponent's response is implicitly absorbed into the transition, so the agent reasons over full move-pairs rather than individual half-turns. This is consistent with the coin game formulation from class and is critical for training stability, since the reward signal is otherwise decoupled from who caused the board change.

Rewards are assigned only at terminal states: +1 for a win, -1 for a loss, and 0 for a draw. All intermediate transitions carry a reward of 0.

Replay Buffer

A replay buffer with a capacity of 20,000 transitions was maintained using a deque. Each transition stores the tuple (state_board, player, action, reward, next_board, done). At each training step, a fixed batch of 64 transitions was sampled uniformly from the buffer. The fixed batch size is an important implementation detail: TensorFlow retriggers graph tracing on variable-length inputs, which would cause training to slow down significantly if batch sizes varied across steps.

A burn-in period of 500 transitions was enforced before any gradient updates were taken. This ensures the initial training batches contain diverse experiences rather than being dominated by the first few games, which tend to be uninformative during early exploration.

Target Network

A separate target network, initialized with identical weights to the online network, was used to compute the Bellman update targets. The target network weights were frozen and updated by copying the online network's weights every 50 episodes. This lag between the online and target networks reduces the instability that arises from chasing a moving target: if the same network were used for both prediction and target computation, a single weight update could destabilize Q-value estimates for nearby states.

The Bellman target for each transition was computed as:

$$y = r \quad \text{(if terminal)}$$

$$y = r + \gamma \cdot max_{a'} Q_{target}(s', a') \qquad \text{(if non-terminal)}$$

Where s' denotes the next state (the board position on the DQN agent's next turn), and a' denotes a feasible action available in the next state, with a discount factor of γ = 0.99. Only legal columns were considered when computing the max Q-value over the next state.

Epsilon-Greedy Exploration and Annealing

Action selection during training followed an epsilon-greedy strategy. With probability epsilon, the agent selected a random legal move; otherwise it selected the column with the highest estimated Q-value. Epsilon was linearly annealed from 0.90 at episode 1 to 0.05 at episode 300, transitioning the agent from 90% exploration at the start to 95% exploitation by the end of training.

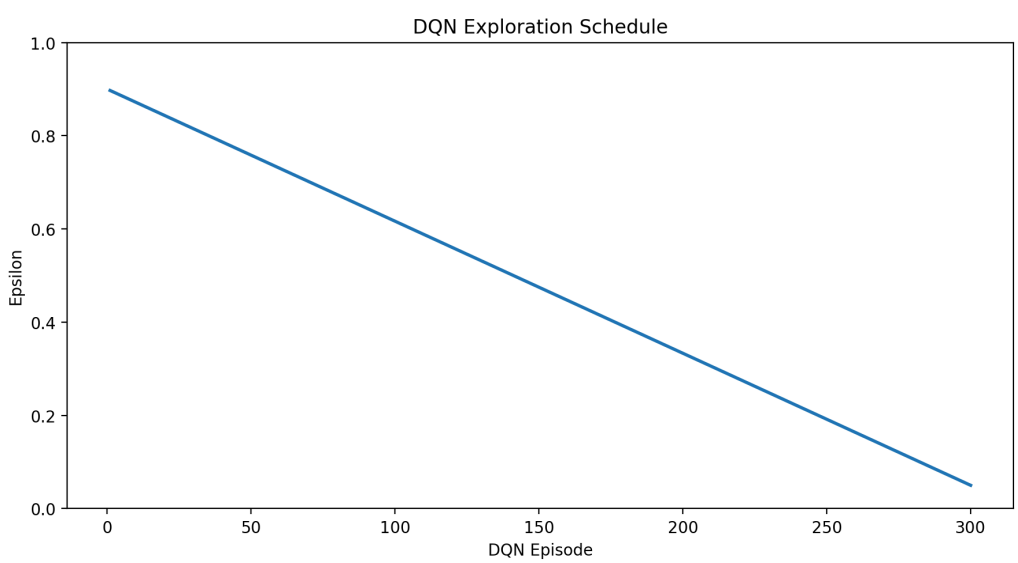

Figure 5. DQN Epsilon-Greedy Exploration Schedule

The high initial epsilon was necessary given the shallow replay buffer and noisy early Q-value estimates. An immediate shift to greedy exploitation would cause the agent to overfit to poor early estimates before enough diverse experiences had been accumulated. As shown in **Figure 5**, the schedule is strictly linear with no plateaus or decay inflection points. A more aggressive decay curve (e.g., exponential) might have benefited convergence speed, and this is noted as an area for improvement if the training budget were expanded.

Additionally, a tactical override was applied for immediate winning moves: when the agent identified a column that would win the game immediately, it would take that move regardless of epsilon. This prevents the agent from randomly passing up trivially available wins during the exploration phase.

Random Board Initialization

Each training game was initialized with 0 to 8 random moves before the DQN began making decisions. This diversifies the board states seen during training: starting from an empty board every game causes the agent to mostly encounter early-game positions and rarely learn from mid-game or late-game scenarios where tactical decisions matter most. Critically, the DQN did not learn from these random opening moves. Only the transitions generated after the random opening were added to the replay buffer.

Opponent Pool

The DQN was trained against a pool of opponents drawn from the three Project 1 baseline models (Riju_CNN, Jake_M1_original, Abhiroop_CNN), each configured with force_win and block_loss tactical helpers to play at full strength. The PG-improved model was also added to the pool, giving the DQN exposure to a policy-based RL opponent as well. A new opponent was selected randomly at the start of each episode, which ensured the DQN was not overfit to any single opponent's style. The DQN was trained for 300 episodes. The project specification notes that 1,000+ episodes would be preferable for full convergence, and the results below should be interpreted in that context.

Deep Q-Network Results and Diagnostics

Training Outcome Trend

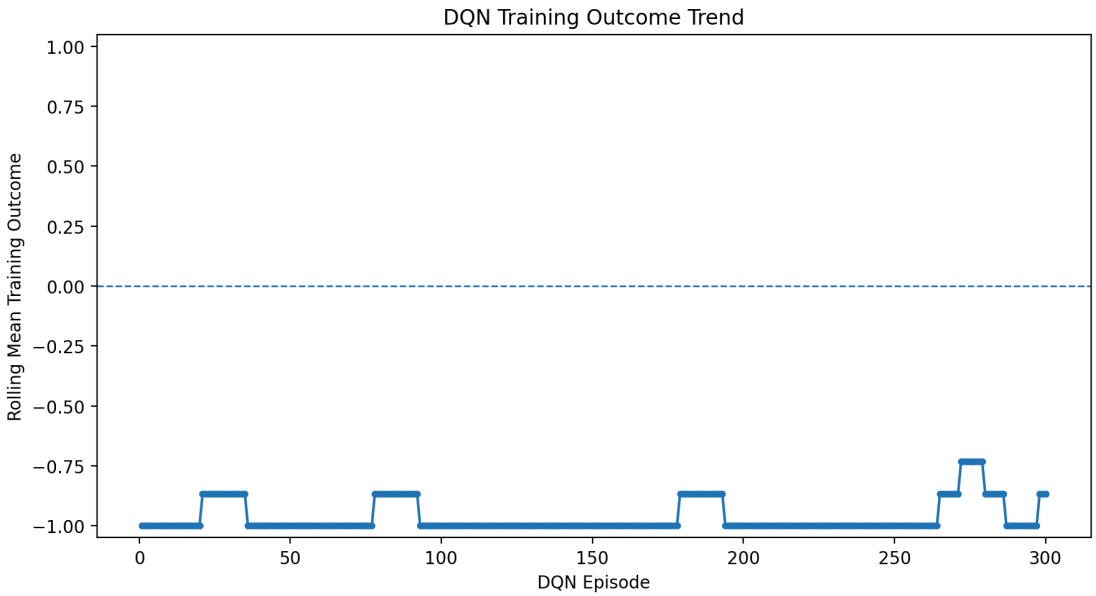

Figure 6. DQN Training Outcome Trend

The DQN training outcome trend in **Figure 6** shows the rolling mean game outcome across 300 episodes. Unlike the PG outcome curve, which fluctuated between roughly -0.5 and +0.4, the DQN curve is almost entirely flat at -1.0 throughout training. Brief deviations toward zero appear near episodes 20-35, 80-90, 175-190, and 260-280. These recoveries correspond to the occasional draws or wins the DQN achieved during high-epsilon periods when random openings happened to produce favorable board states.

The persistent -1.0 rolling mean is not unexpected. The DQN was trained against opponent models that use force_win and block_loss tactical overrides, meaning they always take immediate wins and block immediate losses. An untrained Q-network facing these opponents will lose nearly every game during early training. The improvement seen with PG did not occur here within 300 episodes, reflecting a fundamental difference between the two approaches: PG directly nudges an existing strong policy toward better moves, while DQN must construct an accurate value function from scratch before any performance improvement becomes visible in win rate.

Win Rate Against Original M1 and Random

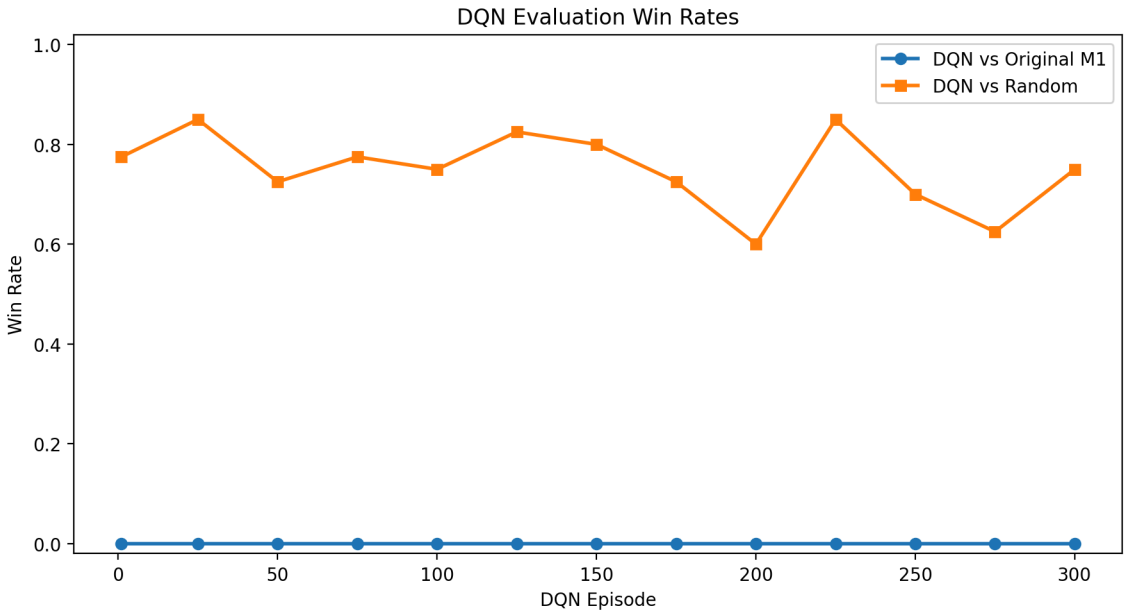

Figure 7. DQN Evaluation Win Rates

The DQN achieved a 0% win rate against the original M1 across all 13 evaluation checkpoints from episode 1 through episode 300. This flat line held without exception, confirming that within the 300-episode training budget, the DQN did not develop the tactical depth required to beat the supervised baseline.

Against the Random opponent, however, the DQN showed meaningful competence. Win rates ranged between 60% and 85% across checkpoints, beginning at 78% at episode 1, peaking at 85% around episodes 25 and 225, and settling near 75% at episode 300. This confirms the DQN learned something beyond random play: it developed a partial sense of which columns offer opportunities and which moves are likely to end badly. However, this level of play was not sufficient to compete with the supervised models.

The contrast between the 0% win rate against M1 and the 60-85% win rate against Random defines the DQN's current capability range: meaningfully above the floor, but well below the ceiling set by the supervised baseline. This gap is consistent with the theoretical expectation that value-based RL methods require substantially more episodes and more diverse experience to converge than policy gradient methods applied to a warm-started pre-trained network.

Loss Curve and Convergence

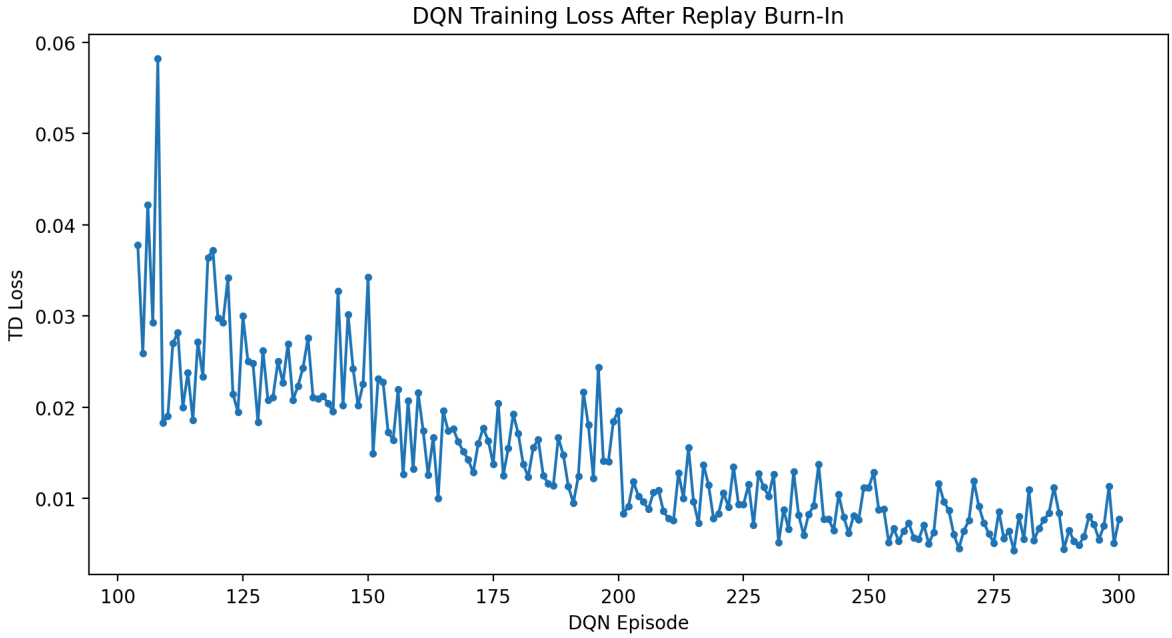

Figure 7. DQN Training Loss After Replay Burn-In

The TD loss after the 500-transition burn-in period begins at roughly 0.038-0.058 around episodes 105-115, reflecting the high initial variance in Q-value estimates. Loss then decreases progressively, settling to a range of approximately 0.006-0.013 by episodes 250-300. This declining trend confirms that the Bellman targets are being fit and that the network is neither diverging nor collapsing.

The declining loss without corresponding improvement in win rate against M1 illustrates a well-known limitation of value-based RL: a well-fitted value function is necessary but not sufficient for strong gameplay. Q-values may be internally consistent (low TD error) while remaining globally miscalibrated relative to what a strong opponent requires. More training episodes with a progressively stronger and more diverse opponent pool would be needed to close this gap.

Comparative Analysis: PG vs DQN vs Baseline

Performance Comparison

The final round-robin tournament provides the most comprehensive comparison across all agents. Each pairing was evaluated over 100 games with alternating move order, covering the original M1 (Riju_CNN), the PG-improved model, the DQN agent, Jake_M1_original, Abhiroop_CNN, and the Random baseline. Some differences between the baseline round-robin results and final round-robin outcomes may reflect the larger evaluation sample in the final comparison (100 games per pair versus 50 in the baseline), which reduces variance in estimated win rates.

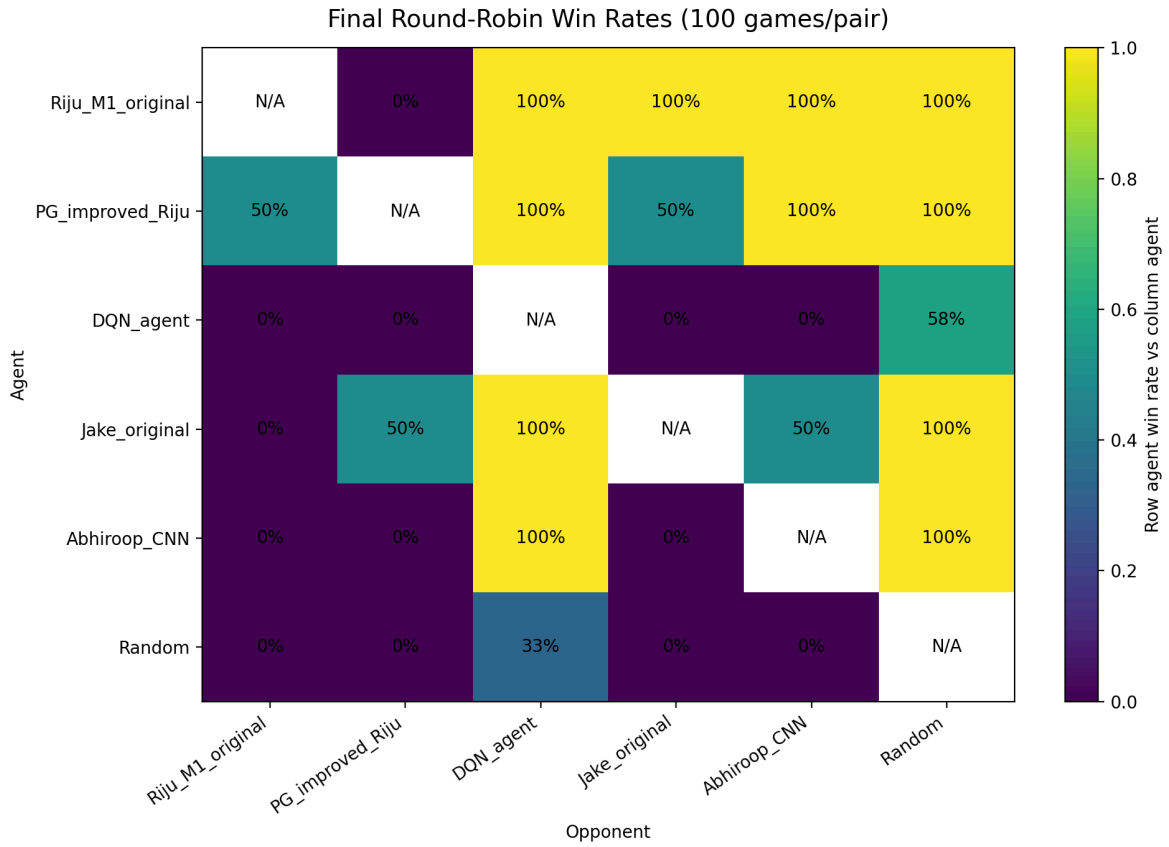

Figure 8. Final Round-Robin Win Rate Heatmap (100 games/pair)

As shown in **Figure 8,** Riju_M1_original is the strongest overall agent, winning 100% of games against Jake_original, Abhiroop_CNN, DQN, and Random, while going 50-50 with the PG-improved model. The PG-improved model wins 50% against both Riju_M1_original and Jake_original, and sweeps Abhiroop_CNN and Random. The DQN agent wins 0% against all trained agents, but manages a 58% win rate against Random.

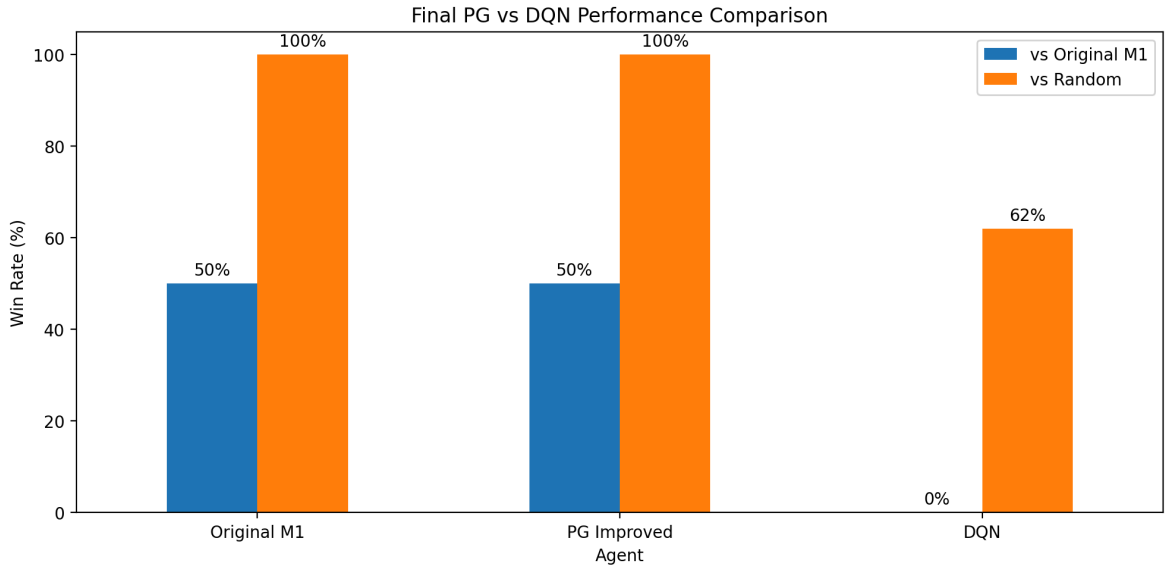

Figure 9. Final Policy Gradient vs DQN Performance Comparison

Head-to-Head Win Rate Summary

Table 5. Head-to-head win rate summary from the final 100-game round-robin evaluation.

Agent	Win Rate vs Original M1	Win Rate vs Random	Final Round-Robin Rank
Original M1 (Riju_CNN)	N/A	100%	1st
PG Improved	50%	100%	2nd
DQN Agent	0%	58% (final RR)	Last among trained models
Jake_M1_original	0%	100%	3rd
Abhiroop_CNN	0%	100%	4th

Move Preference Analysis

Beyond win rates, the move preference chart reveals how each agent's learned strategy differs qualitatively, showing the share of moves each agent played in each of the seven columns over 100 games against a Random opponent.

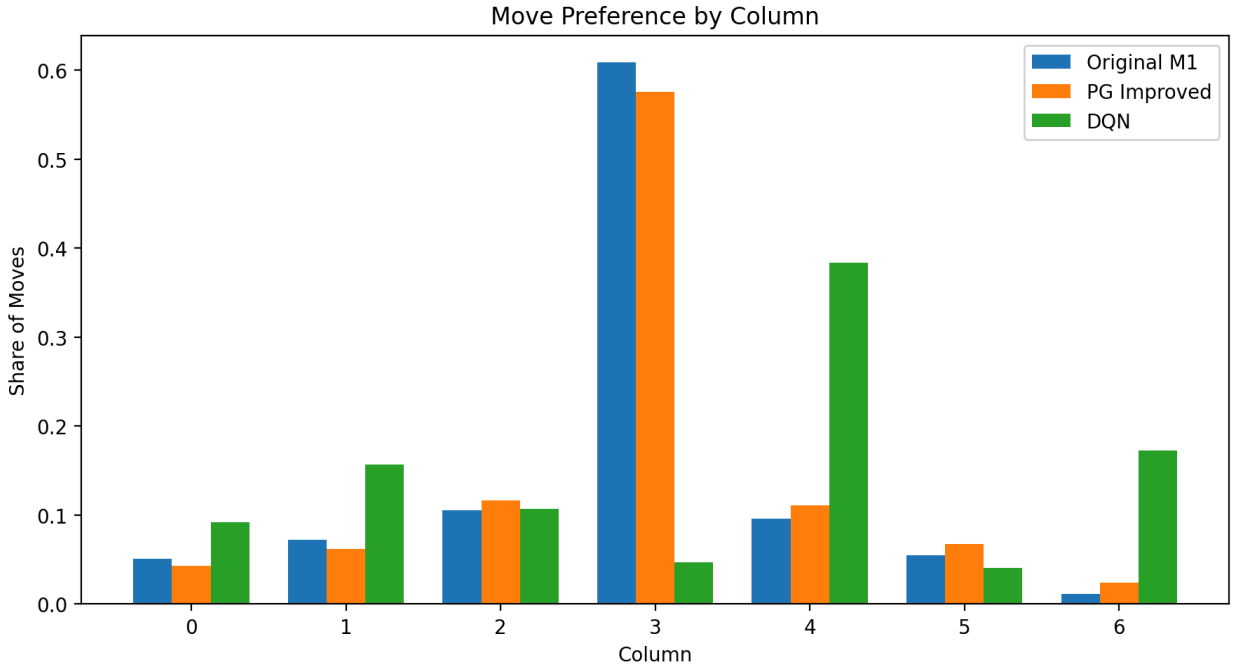

Figure 10. Column Move Preference Distribution by Agent

Original M1 and PG Improved both exhibit a strong center-column preference, placing roughly 61% and 58% of their moves in column 3 respectively. This aligns with established Connect-4 strategy, where the center offers the most connection opportunities across rows and diagonals. The PG model's distribution closely mirrors the original M1's, which is expected since PG begins from M1 and refines it through gradient descent without changing the underlying architecture.

The DQN shows a strikingly different profile. Rather than favoring the center, it assigns the highest share of moves to column 4 (approximately 38%) and column 6 (approximately 17%), while almost entirely avoiding column 3 (approximately 5%). This divergence reflects the fact that the DQN built its value estimates from scratch through Bellman updates, without inheriting the MCTS-distilled center-column preference embedded in the supervised baseline. Whether this represents a genuinely different strategic approach or an artifact of incomplete training is difficult to determine within the current training budget, but it confirms that the two RL approaches converged to qualitatively distinct behaviors.

Computational Tradeoffs

Table 6. Computational and behavioral tradeoffs between the two RL approaches.

Dimension	Policy Gradient (PG)	Deep Q-Network (DQN)
Starting point	Pre-trained M1 (warm start)	Random initialization (cold start)
Training episodes	60 iterations	300 episodes
Convergence speed	Fast: visible improvement against M1 by iteration 35	Slow: 0% win rate against M1 throughout 300 episodes
Update mechanism	REINFORCE: log-probability weighted by discounted outcome	Bellman TD update with target network and replay buffer
Training stability	Medium: win rate swings up to 50 percentage points between evaluations	Low-Medium: outcome trend flat at -1.0 throughout
Random baseline competence	100% from iteration 1 onward	60-85% depending on checkpoint
Implementation complexity	Moderate: extends existing policy network	High: requires target network, replay buffer, burn-in, Bellman update logic
Sample efficiency	Lower: each transition used once	Higher: replay buffer reuses past experiences
Move strategy learned	Center-biased (inherited from M1 and reinforced by PG)	Off-center (columns 4 and 6 dominant)

Which Worked Better and Why

Within the constraints of this project, Policy Gradient clearly outperformed DQN on every measurable criterion. PG achieved a 50-100% win rate against the original M1 during training, a 50% win rate in the final round-robin, and a consistent 100% win rate against Random. DQN achieved 0% against M1 throughout all 300 episodes and only 58-85% against Random.

The reason PG outperformed DQN in this setting comes down to three structural advantages. First, PG started from a warm-initialized policy that already encoded strong Connect-4 priors from the supervised MCTS-imitation training in Project 1. DQN started from random weights with no prior knowledge of the game. This cold start requires the DQN to learn basic game competence from scratch before it can begin to compete with trained opponents, a considerably harder starting position. Second, PG directly modifies the probability of good moves, meaning improvement appears as soon as the gradient updates nudge the policy in the right direction. DQN must first build accurate Q-values for a large and complex state space before its greedy policy becomes competitive, which typically requires an order of magnitude more experience. Third, the 60-iteration budget for PG, while small, was sufficient to produce visible improvement given the warm start. The 300-episode budget for DQN, though five times longer in episode count, was not enough given the cold start and the complexity of learning value estimates from delayed rewards.

That said, PG's advantage is partly a function of the training setup rather than an inherent algorithmic superiority. A DQN trained for 2,000-5,000 episodes, with a progressively expanding opponent pool and curriculum-based difficulty scaling, would likely close the performance gap. The theoretical advantages of value-based RL, including greater sample efficiency from experience replay and the ability to reason explicitly about long-term board value, are real and would become more visible at larger training scales.

Internal Tournament Performance

The evaluation framework simulated tournament-style comparisons using round-robin and head-to-head matches. The baseline results show:

Table 7. Comparative Summary of Agent Performance Across Models

Model	Win Rate vs Other Models	Win Rate vs Random	Observations
Riju_CNN (Baseline M1)	Strongest among baseline models	100%	Chosen as base model
Abhiroop_CNN	Moderate	100%	Competitive but weaker vs Riju
Jake_M1_original	Moderate	100%	Stable but not top-performing
PG Improved Model	Comparable / improved vs baseline M1	100%	Shows learning but fluctuates
DQN Agent	Variable	High	Performance

			depends on training stability

Overall observations:

- All trained models significantly outperform random play, validating baseline competency.
- **Riju_CNN consistently performed best among baseline models**, justifying its selection as M1.
- The **Policy Gradient model showed improvement trends**, occasionally outperforming the original model but with fluctuations due to training variance.
- The **DQN model demonstrated potential**, but results were less consistent due to instability and limited training cycles.

From a tournament-readiness perspective, the improved models are competitive but would benefit from longer training and more stabilization before achieving consistent dominance.

Limitations and Observations

RL training was inherently noisy and unstable, which is expected due to:

- Delayed rewards (only observed at game termination)
- Sensitivity to opponent strength and diversity
- High variance in Policy Gradient updates
- Instability in DQN value estimation, especially early in training

The primary constraint was the **training budget**. More episodes, a larger opponent pool, and deeper evaluation cycles would likely produce more stable and higher-performing agents. As implemented, the results are directionally strong but not fully converged.

Comparative Performance Summary

Table 8. Comparison of Reinforcement Learning Approaches

Approach	Strengths	Limitations	Best Use Case
Policy Gradient (PG)	Directly improves move probabilities; easy integration with supervised models	High variance; sensitive to outcomes	Refining existing policy models
Deep Q-Network (DQN)	Learns long-term value of actions; captures deeper strategy	Requires stabilization (replay, target network); slower convergence	Learning tactical and strategic value
Combined Strategy	Balances policy and value learning	Higher complexity	Production-grade AI systems

Key Insight: PG is more practical for incremental improvement, while DQN is more powerful but requires careful tuning. A hybrid approach would likely outperform both.

Recommendation to Leadership

Should the Company Hire an RL Expert?

Yes - but with a targeted and phased approach.

This project clearly shows that RL has strong potential but also requires careful implementation. Hiring an RL expert would be beneficial in the following areas:

- Designing stable self-play training pipelines
- Structuring and maintaining an effective opponent pool
- Improving reward design and credit assignment
- Implementing advanced methods (actor-critic, hybrid models)
- Establishing rigorous evaluation and tournament frameworks
- Optimizing training efficiency and convergence

However, the recommendation is **not to fully replace existing AI systems immediately**. Instead:

- Start with supervised models as a baseline
- Incrementally improve them using RL techniques
- Validate improvements through controlled evaluation
- Scale only after consistent performance gains are achieved

This minimizes risk while building internal capability.

Business Implications

From a gaming company perspective, RL offers several strategic advantages:

- Enables **adaptive and evolving gameplay**, improving player engagement
- Reduces reliance on labeled data by learning through simulation
- Supports **scalable AI development** across multiple games
- Allows for **difficulty calibration** and personalized player experiences

At the same time, RL introduces operational complexity. Success depends on:

- High-quality training infrastructure
- Strong evaluation pipelines
- Careful tuning of exploration and learning dynamics
- Alignment with user experience goals

Conclusion

Key Outcomes

The most significant outcome is the creation of a robust and reusable RL infrastructure. The system supports legal move masking, stochastic and epsilon-greedy action selection, adversarial self-play, replay-based learning, and structured evaluation through head-to-head matches and tournament-style comparisons. This provides a scalable foundation for experimenting with multiple RL strategies in a consistent and controlled manner.

The Policy Gradient (PG) approach is well aligned with the project requirement of improving a baseline supervised model. By starting from **Riju_CNN** and refining it through self-play, the model learns directly from gameplay outcomes using discounted rewards. This aligns with AlphaGo-style policy improvement, making PG a natural extension of supervised learning into RL.

The Deep Q-Network (DQN) approach complements this by learning action-value functions using Bellman updates. It introduces a stronger notion of long-term planning and benefits from exploration strategies such as epsilon-greedy behavior and random board initializations, which expose the agent to diverse and strategically meaningful states.

Final Recommendation

Reinforcement learning is a **high-potential but execution-sensitive capability**. This project validates that:

- RL can improve gameplay agents beyond supervised baselines
- Policy Gradient provides a stable path for incremental improvement
- DQN enables deeper strategic learning but requires more tuning
- A hybrid approach is likely the best long-term solution

The recommended path forward is:

1. Use supervised models as the foundation
2. Apply Policy Gradient methods for initial improvement
3. Introduce value-based methods (DQN or actor-critic) for deeper optimization
4. Invest in training infrastructure and evaluation frameworks
5. Gradually scale RL across products

With the right expertise and investment, RL can become a **core capability for building intelligent, scalable, and engaging game AI systems**.